

JAL2301 DATA SCIENCE

UNIT IV PYTHON LIBRARIES FOR DATA WRANGLING



UNIT IV PYTHON LIBRARIES FOR DATA WRANGLING SYLLABUS

Basics of Numpy arrays – Aggregations
– Computations on arrays –
Comparisons, masks and boolean
logic – Fancy indexing – Sorting arrays
- Structured data

Data manipulation with Pandas – Data
indexing and selection – Operating on
data – Handling missing data –
Hierarchical indexing – Combining
datasets – Pivot tables.





DATA WRANGLING

DATA WRANGLING

- World of data rapidly expanding → essential to get the right data to be organized for analysis.
- Business users rely on data and information to make just about every business decision.
- Hence it is important to make raw data usable for analytics.
- Rapid expansion of the amount of data and data sources available today → hence storing and organizing large quantities of data for analysis is becoming increasingly necessary.

DATA WRANGLING

Different definitions for Data Wrangling

- Data wrangling (or data munging) → the process of removing errors and combining complex data sets to make them more accessible and easier to analyze.
- Data wrangling → the process of converting and mapping raw data and getting it ready for analysis.
- Data wrangling → defined as the process of cleaning, organizing and transforming raw data into the desired format for analysts to use for prompt decision-making.
- Data wrangling process consists of reorganizing, transforming and mapping data from one "raw" form into another in order to make it more usable and valuable for a variety of downstream uses including analytics.
- Data wrangling enables businesses to tackle more complex data in less time, produce more accurate results, and make better decisions.

5



BASICS OF NUMPY ARRAYS

BASICS OF NUMPY ARRAYS

1. NUMPY ARRAY ATTRIBUTES

- RANDOM NUMBER GENERATOR
- SEED
- ARANGE(), NDIM(), SHAPE(), SIZE(), DTYPE(), ITEMSIZE(), NBYTES()

2. ARRAY INDEXING - ACCESSING SINGLE ELEMENTS

3. ARRAY SLICING - ACCESSING SUBARRAYS

- COPY()

4. RESHAPING OF ARRAYS

- RESHAPE()

5. ARRAY CONCATENATION AND SPLITTING

- CONCATENATE(), VSTACK(), HSTACK()
- SPLIT(), VSPLIT(), HSPLIT()

7

BASICS OF NUMPY ARRAYS

- NumPy array manipulation used
 - To access data and subarrays and
 - To split, reshape and join the arrays

8

BASICS OF NUMPY ARRAYS

- Few categories of basic array manipulations:

Array Manipulations	Use
Attributes of arrays	Determining the size, shape, memory consumption and data types of arrays
Indexing of arrays	Getting and setting the value of individual array elements
Slicing of arrays	Getting and setting smaller subarrays within a larger array
Reshaping of arrays	Changing the shape of a given array
Joining and splitting of arrays	Combining multiple arrays into one and splitting one array into many

NUMPY ARRAY ATTRIBUTES

Common attributes and functions associated with NumPy arrays in Python, often used to manipulate and analyze array data

- `arange` → generates a sequence of values from start to stop with a given step size
- `ndim` → returns the number of dimensions
- `shape` → returns the size of each dimension
- `size` → returns the total size of the array
- `dtype` → returns the data type of the array
- `itemsize` → returns the size (in bytes) of each array element
- `nbytes` → returns the total size (in bytes) of the array

1. NUMPY ARRAY ATTRIBUTES

- NumPy array manipulation uses **NumPy's random number generator** → seed with a set value in order to ensure that the same random arrays are generated each time this code is run
- **Seed method** → used to initialize the random number generator → provides a seed value → the generator starts with a set value in order to ensure that the same random arrays are generated each time this code is run



11

1. NUMPY ARRAY ATTRIBUTES

The function
name

The shape of
the output

```
np.random.randint(low, high=, size=, dtype=)
```

The lowest
possible
integer

The highest
possible
integer

The
datatype of
the output



12

1. NUMPY ARRAY ATTRIBUTES

- **NumPy's random number generator** → seed with a set value in order to ensure that the **same random arrays** are generated each time this code is run

```
import numpy as np
np.random.seed(0) # seed for reproducibility
x1 = np.random.randint(10, size=6) # One-dimensional array
x2 = np.random.randint(10, size=(3, 4)) # Two-dimensional array
x3 = np.random.randint(10, size=(3, 4, 5)) # Three-dimensional array
# same arrays generated when seed method is used
```

13

1. NUMPY ARRAY ATTRIBUTES

OUTPUT:

X1

[5 0 3 3 7 9]

OUTPUT:

X2

[[3 5 2 4]

[7 6 8 8]

[1 6 7 7]]

OUTPUT:

X3

[[[8 1 5 9 8]

[9 4 3 0 3]

[5 0 2 3 8]

[1 3 3 3 7]]

[[[0 1 9 9 0]

[4 7 3 2 7]

[2 0 0 4 5]

[5 6 8 4 1]]

[[[4 9 8 1 1]

[7 9 9 3 6]

[7 2 0 3 5]

[9 4 4 6 4]]]

14

1. NUMPY ARRAY ATTRIBUTES

- `arange` → generates a sequence of values from start to stop with a given step size
- `ndim` → the number of dimensions
- `shape` → the size of each dimension
- `size` → the total size of the array
- `dtype` → the data type of the array
- `itemsize` → the size (in bytes) of each array element
- `nbytes` → the total size (in bytes) of the array

```
X4 = np.arange(0, 10, 2)
print(X4)
print("x3 ndim:", x3.ndim)
print("x3 shape:", x3.shape)
print("x3 size:", x3.size)
print("dtype:", x3.dtype)
print("itemsize:", x3.itemsize, "bytes")
print("nbytes:", x3.nbytes, "bytes")
```

OUTPUT:
 [0 2 4 6 8]
 x3 ndim: 3
 x3 shape: (3, 4, 5)
 x3 size: 60
 dtype: int64
 itemsize: 8 bytes
 nbytes: 480 bytes

15

2. ARRAY INDEXING - ACCESSING SINGLE ELEMENTS

- Access the i^{th} value (counting from zero) by specifying the desired index in square brackets.
- In 1-d arrays:

```
print(x1)
print(x1[0])
print(x1[-2])
x1[2] = 6.1754
print(x1)
```

OUTPUT:
 [5 0 3 3 7 9]
 5
 7
 [5 0 6 3 7 9]

16

2. ARRAY INDEXING - ACCESSING SINGLE ELEMENTS

- In 2-d arrays:

```
print(x2)
print(x2[0, 0])
print(x2[2, 0])
print(x2[2, -1])
x2[0, 0] = 30
print(x2)
```

OUTPUT:

```
[[3 5 2 4]
 [7 6 8 18]
 [1 6 17 27]]
```

```
3
```

```
1
```

```
27
```

```
[[30 5 2 4]
 [7 6 8 18]
 [1 6 17 27]]
```

3. ARRAY SLICING - ACCESSING SUBARRAYS

- Use [] to access subarrays with the slice notation marked by the colon (:) character.
- **x[start : stop : step]**
- Default values: start → 0 stop → size of dimension step=1

3. ARRAY SLICING - ACCESSING SUBARRAYS

■ In 1-d arrays:

```
x = np.arange(15, 27)
print(x)
print(x[:5])
print(x[5:])
print(x[4:7])
print(x[: 2])
print(x[1: 2])
print(x[: : -1]) # all elements
reversed
print(x[5: : -2]) # reversed every
other from index 5
```

OUTPUT:

```
[15 16 17 18 19 20 21 22 23
24 25 26]
[15 16 17 18 19]
[20 21 22 23 24 25 26]
[19 20 21]
[15 17 19 21 23 25]
[16 18 20 22 24 26]
[26 25 24 23 22 21 20 19 18
17 16 15]
[20 18 16]
```

3. ARRAY SLICING - ACCESSING SUBARRAYS

■ In 2-d arrays:

```
print(x2)
print("\nAAA = ", x2[: 2, : 3]) # two rows three
columns
print("\nBBB = ", x2[:3, :: 2]) #all rows every other
column
print("\nCCC = ", x2[: : -1, :: ]) #all rows reversed
print("\nDDD = ", x2[: : -1, :: -1]) #all elements
reversed
```

OUTPUT:

```
[[12 5 2 4]
 [ 7 6 8 8]
 [ 1 6 7 7]]
AAA = [[12 5 2]
        [ 7 6 8]]
BBB = [[12 2]
        [ 7 8]
        [ 1 7]]
CCC = [[ 1 6 7 7]
        [ 7 6 8 8]
        [12 5 2 4]]
DDD = [[ 7 7 6 1]
        [ 8 8 6 7]
        [ 4 2 5 12]]
```

3. ARRAY SLICING - ACCESSING SUBARRAYS

Accessing array rows and columns

- Commonly needed routine → for accessing single row or column of an array → done by combining indexing and slicing

```
print(x2)
print("A = ", x2[:, 0]) # first column of x2
print("B = ", x2[0, :]) # first row of x2
print("C = ", x2[0]) # equivalent to x2[0, :]
```

OUTPUT:

```
[[12 5 2 4]
 [ 7 6 8 8]
 [ 1 6 7 7]]
A = [12 7 1]
B = [12 5 2 4]
C = [12 5 2 4]
```

3. ARRAY SLICING - ACCESSING SUBARRAYS

Subarrays as no-copy views

- Array slicing → **return views** (not copies) of the array data.
- List slicing → slices will be copies
- Advantage: access and process pieces of these datasets without the need to copy the underlying data buffer

```
print("Original = ", x2)
B = x2[:2, :2]
print("B = ", B) # subarray of x2
B[0, 0] = 99
print("Original = ", x2)
```

OUTPUT:

```
Original = [[3  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
B = [[3 5]
 [ 7 6]]
Original = [[99 5 2 4]
 [ 7 6 8 8]
 [ 1 6 7 7]]
```

3. ARRAY SLICING - ACCESSING SUBARRAYS

Creating copies of arrays

- Using `copy()` method
- Modification to the copy of subarray does not cause change in the original array

```
print("Original = ", x2)
arr_copy = x2[:2, :2].copy()
print("COPY = ", arr_copy)
arr_copy[0, 0] = 75
print("NEW COPY = ", arr_copy)
print("Original = ", x2)
```

OUTPUT:

```
Original = ([[3 5 2 4]
             [ 7 6 8 8]
             [ 1 6 7 7]])
COPY = [[3 5]
        [ 7 6]]
NEW COPY = [[75 5]
            [ 7 6]]
Original = ([[3 5 2 4]
             [ 7 6 8 8]
             [ 1 6 7 7]])
```

4. RESHAPING OF ARRAYS

METHOD 1:

- Using `reshape()` method
- Eg. to put the numbers 1 through 9 in a 2-d grid
- Size of the initial array must match the size of the reshaped array.

```
a = np.arange(1, 10).reshape((3, 3))
print("A = ", a)

b = np.arange(1, 15).reshape((3, 3))
print("B = ", b)
```

OUTPUT:

```
A = [[1 2 3]
     [ 4 5 6]
     [ 7 8 9]]
```

```
ValueError: cannot
reshape array of size 14
into shape (3,3)
```

4. RESHAPING OF ARRAYS

METHOD 2:

- Conversion of a one-dimensional array into a two-dimensional row or column matrix.

```
b = np.array([1, 32, 73, 44, 50, 10]).reshape((2, 3))
print("B = ", b)
```

OUTPUT:

```
B = [[1 32 73]
      [ 44 50 10]]
```

25

5. ARRAY CONCATENATION AND SPLITTING

- It is possible to combine multiple arrays into one and to split a single array into multiple arrays.

Concatenation of Arrays

- Using concatenate() method
- Using vstack() method
- Using hstack() method
- For concatenating 1-d arrays:

OUTPUT:

```
NEW = [ 1  8  5  9  6  7 ]
NEW1 = [ 1  8  5  9  6  7 99 89 69]
```

```
a = np.array([1, 8, 5])
b = np.array([9, 6, 7])
print("NEW = ", np.concatenate([a, b]))
c = [99, 89, 69]
print("NEW1 = ", np.concatenate([a, b, c]))
```

26

5. ARRAY CONCATENATION AND SPLITTING

Concatenation of Arrays

- For concatenating 2-d arrays:

```
arr1 = np.array([[1, 2, 3], [4, 5, 6]])
#concatenate along the first axis
print("NEW = ", np.concatenate([arr1, arr1]))

#concatenate along the second axis
print("NEW1 = ", np.concatenate([arr1, arr1],
axis=1))
```

OUTPUT:

```
NEW = [[1 2 3]
 [4 5 6]
 [1 2 3]
 [4 5 6]]
```

```
NEW1 = [[1 2 3 1 2 3]
 [4 5 6 4 5 6]]
```

27

5. ARRAY CONCATENATION AND SPLITTING

Concatenation of Arrays

- For concatenating arrays of mixed dimensions:
- Using **vstack()** (vertical stack) → to concatenate arrays with same no. of **columns**
 - Eg. concatenate 1 x 3 array and 2 x 3 array
- **hstack()** (horizontal stack) → to concatenate arrays with same no. of **rows**
 - Eg. concatenate 3 x 1 array and 3 x 5 array

```
arr1 = np.array([1, 2, 3]) #shape 1 x 3
arr2 = np.array([[10, 20, 30], [40, 50, 60]])
arr3 = np.array([[80], [90]]) #shape 2 x 1
#vertically stack the arrays
print("NEW1 = ", np.vstack([arr1, arr2]))
#horizontally stack the arrays
print("NEW2 = ", np.hstack([arr2, arr3]))
```

OUTPUT:

```
NEW1 = [[ 1  2  3]
 [10 20 30]
 [40 50 60]]
```

```
NEW2 = [[10 20 30 80]
 [40 50 60 90]]
```

5. ARRAY CONCATENATION AND SPLITTING

Splitting of Arrays

- A list of indices along with the split points is passed.
- Functions used.
 - Using `split()` method
 - Using `vsplit()` method
 - Using `hsplit()` method
- For splitting 1-d arrays: n split points used to get n+1 subarrays

```
a = np.array([1, 8, 5, 9, 6, 7, 12, 3])
x1, x2, x3 = np.split(a, [3, 5])
print(x1, "\n", x2, "\n", x3)
x4, x5, x6 = np.split(a, [3, 8])
print("\n", x4, x5, x6)
```

OUTPUT:

```
[1 8 5]
[9 6]
[7 12 3]
[1 8 5] [9 6 7 12 3] []
```

5. ARRAY CONCATENATION AND SPLITTING

Splitting of Arrays

- For splitting 2-d arrays: use `vsplit()` and `hsplit()`
- `vsplit()` gives arrays with same no. of columns
- `hsplit()` gives arrays with same no. of rows

```
a = np.arange(16).reshape((4, 4))
print(a)
x1, x2 = np.vsplit(a, [3]) #split from 3rd row
print(x1, "\n", x2)

x3, x4 = np.hsplit(a, [3])
print(x3, "\n", x4)
```

OUTPUT:

```
[[ 0, 1, 2, 3],
 [ 4, 5, 6, 7],
 [ 8, 9, 10, 11],
 [12, 13, 14, 15]]

[[ 0, 1, 2],
 [ 4, 5, 6],
 [ 8, 9, 10]]
[[12, 13, 14]]

[[ 0, 1, 2, 3],
 [ 4, 5, 6, 7],
 [ 8, 9, 10, 11],
 [12, 13, 14, 15]]

[[ 3]
 [ 7]
 [11]
 [15]]
```



AGGREGATIONS

AGGREGATIONS

- **SUMMING THE VALUES IN AN ARRAY**
 - `SUM()`
- **MINIMUM AND MAXIMUM**
 - `MIN()`, `MAX()`
- **MULTIDIMENSIONAL AGGREGATES**
- **OTHER AGGREGATION FUNCTIONS**
 - **NAN-SAFE FUNCTIONS**
- **EXAMPLE: WHAT IS THE AVERAGE HEIGHT OF PERSONS?**

AGGREGATIONS

- Large amount of data – the 1st step → computation of summary statistics for the data
- Like mean and standard deviation, sum, product, median, minimum and maximum, quantiles, etc.
- NumPy has fast built-in aggregation functions for working on arrays



33

SUMMING THE VALUES IN AN ARRAY

- Using the built-in sum function – `np.sum()`
- `np.sum` is aware of multiple array dimensions

```
float_arr1 = np.random.random(10)
print(float_arr1)
S = np.sum(float_arr1)
print("SUM1 = %.2f" % S)
```

```
float_arr2 = np.random.uniform(10, size=5)
print(float_arr2)
print("SUM2 = ", np.sum(float_arr2))
```

```
int_arr = np.random.randint(30, size=10)
print(int_arr)
print("SUM3 = ", np.sum(int_arr))
```

OUTPUT:

```
[0.64944802 0.75286113
 0.31502513 0.21617938
 0.81383089 0.36865529
 0.29930838 0.32279162]
SUM1 = 3.74
```

```
[5.5525766 9.83781188
 8.8132604 1.92440865
 5.70297681]
```

```
SUM2 = 31.83103431801866
[18 26 21 12 22 12 19 19 10 3]
SUM3 = 162
```

MINIMUM AND MAXIMUM

- To find the minimum value and maximum value of any given array:-
np.min() and np.max()

```
float_arr1 = np.random.random(10)
print(float_arr1)
print("%.2f " % np.min(float_arr1), "%.4f " % np.max(float_arr1))
```

OUTPUT:

```
[0.86371093 0.40437805 0.07562084 0.99811362 0.13879137
0.76615884 0.18589511 0.0108567 0.05266977 0.83861206]

0.01 0.9981
```

MULTIDIMENSIONAL AGGREGATES

- The axis keyword specifies the dimension of the array that will be collapsed

```
M = np.random.random((3, 4))
print(M)
print("\n SUM = ", np.sum(M))
mini = np.min(M, axis=0) # minimum value within each column
print("MINIMUM = ", mini)
print("MAXIMUM = ", np.max(M, axis=1) # maximum value within each row
```

OUTPUT:

```
[[0.96613324 0.34407034 0.37472459 0.0735107 ]
 [0.85412972 0.70257661 0.35616093 0.51967225]
 [0.44429181 0.86276412 0.84862076 0.08229404]]

SUM = 6.428949112091967
MINIMUM = [0.44429181 0.34407034 0.35616093 0.0735107 ]
MAXIMUM = [0.96613324 0.85412972 0.86276412]
```

OTHER AGGREGATION FUNCTIONS

- NaN-safe functions → compute the result while ignoring missing values

Aggregation functions available in NumPy:

Function Name	NaN-safe Version	Description
<code>np.sum</code>	<code>np.nansum</code>	Compute sum of elements
<code>np.prod</code>	<code>np.nanprod</code>	Compute product of elements
<code>np.mean</code>	<code>np.nanmean</code>	Compute median of elements
<code>np.std</code>	<code>np.nanstd</code>	Compute standard deviation
<code>np.var</code>	<code>np.nanvar</code>	Compute variance
<code>np.min</code>	<code>np.nanmin</code>	Find minimum value
<code>np.max</code>	<code>np.nanmax</code>	Find maximum value
<code>np.argmin</code>	<code>np.nanargmin</code>	Find index of minimum value
<code>np.argmax</code>	<code>np.nanargmax</code>	Find index of maximum value
<code>np.median</code>	<code>np.nanmedian</code>	Compute median of elements
<code>np.percentile</code>	<code>np.nanpercentile</code>	Compute rank-based statistics of elements
<code>np.any</code>	N/A	Evaluate whether any elements are true
<code>np.all</code>	N/A	Evaluate whether all elements are true

EXAMPLE: WHAT IS THE AVERAGE HEIGHT OF PERSONS?

- The file heights.csv has the following columns: s_no, name, height(cm)
- Apply aggregates and find the average height of all the persons.
- Use pandas library function to read csv file.

```
import numpy as np
import pandas as pd
path = 'c:/data/heights.csv'
data = pd.read_csv(path)
heights = np.array(data['height(cm)'])
print(heights)
print("Mean height: ",
      np.mean(heights))
```

OUTPUT:

```
[165 157 149 162 167 161
 147 146 180 150 169 167 169
 146 149]
Mean height:
158.93333333333334
```

EXAMPLE: WHAT IS THE AVERAGE HEIGHT OF PERSONS?

- A variety of summary statistics can be computed.
- Aggregates give us information about the distribution of values

```
import numpy as np
import pandas as pd
path = 'c:/data/heights.csv'
data = pd.read_csv(path)
heights = np.array(data['height(cm)'])
print(heights)
print("Mean height: ", "%.3f"
      %np.mean(heights))
print("Variance: ", "%.3f" %np.var(heights))
print("Standard deviation:", "%.3f"
      %np.std(heights))
print("Minimum height: ", np.min(heights))
print("Maximum height: ", np.max(heights))
```

OUTPUT:

```
[165 157 150 162 167 161
 147 145 180 154 169 167
 169 146 153]
Mean height: 159.467
Variance: 97.982
Standard deviation: 9.899
Minimum height: 145
Maximum height: 180
```

EXAMPLE: WHAT IS THE AVERAGE HEIGHT OF PERSONS?

- Computation of quartiles

```
import numpy as np
import pandas as pd
path = 'c:/data/heights.csv'
data = pd.read_csv(path)
heights = np.array(data['height(cm)'])
print(heights)
print("25th percentile: ",
      np.percentile(heights, 25))
print("Median: ", np.median(heights))
print("75th percentile: ",
      np.percentile(heights, 75))
```

OUTPUT:

```
[165 157 150 162 167 161
 147 145 180 154 169 167
 169 146 153]
25th percentile: 151.5
Median: 161.0
75th percentile: 167.0
```

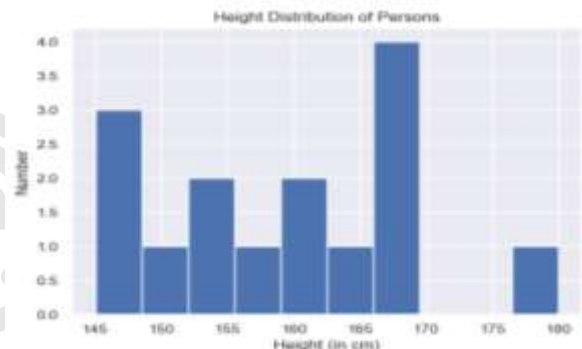
EXAMPLE: WHAT IS THE AVERAGE HEIGHT OF PERSONS?

- Visual representation of this data can be accomplished using tools in Matplotlib and seaborn
- Seaborn is a library for making statistical graphics in Python. It builds on top of matplotlib and integrates closely with pandas data structures.

```
import matplotlib.pyplot as plt
import seaborn as sea
sea.set()
print(heights)
plt.hist(heights)
plt.title('Height Distribution of
Persons')
plt.xlabel('Height (in cm)')
plt.ylabel('Number')
plt.show()
```

OUTPUT:

```
[165 157 150 162 167 161 147
145 180 154 169 167 169 146
153]
```



COMPUTATIONS ON ARRAYS

COMPUTATIONS ON ARRAYS

- **BROADCASTING**
 - ADDING 1-D ARRAY WITH A SCALAR
 - ADDING 1-D ARRAY & A 2-D ARRAY
 - BROADCASTING 2 ARRAYS
- **RULES OF BROADCASTING**
- **EXAMPLES FOR BROADCASTING**

43

UFUNC IN NUMPY

- Ufunc → means universal functions
- ufuncs are used to implement vectorization in NumPy which is way faster than iterating over elements.
- Vectorization → the process of converting an algorithm from operating on a single value at a time to operating on a set of values (vector) at one time → operations on arrays can be performed without using loops.

44

ARITHMETIC OPERATORS IMPLEMENTED IN NUMPY USING UFUNC

Operator	Ufunc	Description
+	np.add	Addition
-	np.subtract	Subtraction
-	np.negative	Unary negation
*	np.multiply	Multiplication
/	np.divide	Division
//	np.floor_divide	Floor division
**	np.power	Exponentiation
%	np.mod	Modulus/remainder

45

ARITHMETIC OPERATORS IMPLEMENTED IN NUMPY USING UFUNC

Function Type	Ufunc	Description
Absolute value	np.abs, np.absolute	np.abs(-5) = 5 np.abs(3-4j) = 5.0 $\rightarrow \sqrt{3^2 + 4^2}$
Trigonometric	np.sin, np.cos, np.tan	np.sin(0) = 0
Exponents	np.exp, np.exp2, np.power	np.exp(1) = 2.71828183 $\rightarrow e^1$ np.exp2(3) = 8 $\rightarrow 2^3$ np.power(3, 2) = 9 $\rightarrow 3^2$
Logarithmic	np.log, np.log2, np.log10	np.log(2) = 0.69314718 $\rightarrow \ln(2)$ np.log2(8) = 3 $\rightarrow \log_2 8$ np.log10(10) = 1 $\rightarrow \log_{10} 10$

46

BROADCASTING

- One vectorizing operation → to use NumPy's broadcasting functionality.
- Broadcasting is simply a set of rules for applying binary ufuncs (addition, subtraction, multiplication, etc.) on arrays of different sizes.

47

BROADCASTING – ADDING 1-D ARRAY WITH A SCALAR

- For arrays of the same size, operations are performed on an element-by-element basis:

```
import numpy as np
a = np.array([16, 35, 21])
b = np.array([5, 5, 5])
print(a + b)
```

OUTPUT:
[21 40 26]

- Broadcasting allows these types of operations to be performed on arrays of different sizes

```
print(a + 5)
```

OUTPUT:
[21 40 26]

48

BROADCASTING – ADDING 1-D ARRAY WITH A 2-D ARRAY

- Broadcasting → stretching or duplicating the value 5 into the array [5, 5, 5]
- This can be extended to arrays of higher dimension

1-d array 'a' is stretched, or broadcast, across the 2nd Dimension (column) in order to match the shape of c

```
import numpy as np
a = np.array([0, 1, 2])
b = np.ones((3, 3))
c = np.ones((3, 3), dtype='int')

print(b)
print(b + 5)

print(c)
print(c + a)
```

OUTPUT:

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [[6. 6. 6.]
 [6. 6. 6.]
 [6. 6. 6.]
 [[1 1 1]
 [1 1 1]
 [1 1 1]]
 [[1 2 3]
 [1 2 3]
 [1 2 3]]
```

BROADCASTING – 2 ARRAYS

- newaxis() → numpy function used to increase the dimension of the existing array by one more dimension

1-d array is increased into a 2-d array

```
import numpy as np
a = np.array([10, 21, 12, 7])
print("A = ", a)
b = a[np.newaxis, :]
print("B = ", b)
print(np.shape(b))
c = a[:, np.newaxis]
print("C = ", c)
print(np.shape(c))
print(b + c) #refer next slide
```

OUTPUT:

```
A = [10 21 12 7]
B = [[10 21 12 7]]
(1, 4)
C = [[10]
 [21]
 [12]
 [ 7]]
(4, 1)
```

BROADCASTING – 2 ARRAYS

- Adding 2 arrays of different shapes

Both of the 2-d arrays b and c are stretched, to a common shape to result in a 2-d array

```
print(b + c)
```

OUTPUT:

```
[[20 31 22 17]
 [31 42 33 28]
 [22 33 24 19]
 [17 28 19 14]]
```

	b	c
	10 21 12 7	10 10 10 10
	10 21 12 7	21 21 21 21
	10 21 12 7	12 12 12 12
	10 21 12 7	7 7 7 7
Arrays broadcasted as given		

51

BROADCASTING – 2 ARRAYS

```
a = np.arange(3)
print(a)
print(a+5)
b = np.ones((3, 3))
print(b)
print(a + b)
c = np.arange(3).reshape(3, 1)
print(c)
print(a + c)
```

OUTPUT:

```
[0 1 2]
[5 6 7]
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
[[1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]]
[[0]
 [1]
 [2]]
[[0 1 2]
 [1 2 3]
 [2 3 4]]
```

RULES OF BROADCASTING

- **Rule 1:** If the two arrays differ in their number of dimensions, the shape of the one with fewer dimensions is padded with ones on its leading (left) side.
 - (Eg. when adding 1-d array with a 2-d array, the 1-d array with size 3 is converted into a 2-d array with shape 1,3)
- **Rule 2:** If the shape of the two arrays does not match in any dimension, the array with shape equal to 1 in that dimension is stretched to match the other shape.
- **Rule 3:** If in any dimension the sizes disagree and neither is equal to 1, an error is raised.

53

RULES OF BROADCASTING

Example: adding 1-d array with 2-d array

- `M = np.ones((2, 3))`
- `a = np.arange(3)`
- `M.shape → (2, 3)`
- `a.shape → (3,)`
- By Rule 1, since array 'a' has fewer dimensions, we pad it on the left with ones:
- `a = a[np.newaxis, :] → a.shape → (1, 3)`
- `print("Shape of M = ",` By rule 2, since the first dimension disagrees, stretch this dimension to match:
- `a.shape → (2, 3)`
- `M + a`

`M = [[1 1 1]
[1 1 1]]`

`a = [0 1 2]`

After padding,
`a = [[0 1 2]]`

`M+a =
[[1 2 3]
[1 2 3]]`

`[[1 1 1] [0 1 2] [[1 2 3]
[1 1 1]] + [0 1 2] = [1 2 3]]`

54

RULES OF BROADCASTING

Example: both arrays need to be broadcast

- `a = np.arange(3).reshape((3, 1))`
- `b = np.arange(3)`
- `a.shape = (3, 1)`
- `b.shape = (3,)`
- By Rule 1 → pad the shape of b with ones on the left:
- `a.shape → (3, 1)`
- `b[np.newaxis, :] → b.shape → (1, 3)`
- By Rule 2 → upgrade each of these ones to match the corresponding size of the other array:
- `a.shape → (3, 3)`
- `b.shape → (3, 3)`
- `a + b`

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 2 & 3 \\ 2 & 3 & 4 \end{bmatrix}$$

`a = [[0]
[1]
[2]]`

`b = [0 1 2]`

After padding,
`b = [[0 1 2]]`

`SUM = [[0 1 2]
[1 2 3]
[2 3 4]]`



COMPARISONS,
MASKS, BOOLEAN
LOGIC

COMPARISONS, MASKS, BOOLEAN LOGIC

- **COMPARISON OPERATORS AS UFUNCS**
- **WORKING WITH BOOLEAN ARRAYS**
 - COUNTING ENTRIES
 - BOOLEAN OPERATORS
- **BOOLEAN ARRAYS AS MASKS**

57

COMPARISON OPERATIONS

- Masking comes up when we want to extract, modify, count, or otherwise manipulate values in an array based on some criterion: for example,
 - Count all values greater than a certain value, or
 - Remove all outliers that are above some threshold.
- This needs
 - comparison,
 - applying 1 or more conditions (or criteria) and
 - selecting only few values (ignoring others)
- NumPy implements comparison operators such as $<$ (less than) and $>$ (greater than) as element-wise ufuncs – output $\rightarrow$ Boolean array
- Also possible to include compound expressions

58

COMPARISON OPERATIONS

NumPy implements comparison operators such as < (less than) and > (greater than) as element-wise ufuncs – output → Boolean array

```
a = np.array([1, 2, 3, -4, 5])
print(a < 3)
print(a > 3)
print(a <= 3)
print(a >= 3)
print(a != 3)
print(a == 3)

print((2 * a) == (a ** 2))
```

OUTPUT:

```
[ True  True False  True False]
[False False False False  True]
[ True  True  True  True False]
[False False  True False  True]
[ True  True False  True  True]
[False False  True False False]

[False  True False False False]
```

COMPARISON OPERATORS IMPLEMENTED IN NUMPY USING UFUNC

Comparison Operators	Equivalent Ufunc	<pre>import numpy as np x2 = np.random.randint(10, size=(2, 3)) print(x2) print(x2 < 4)</pre>
==	np.equal	
!=	np.not_equal	
<	np.less	
<=	np.less_equal	
>	np.greater	
>=	np.greater_equal	

Output:

```
[[9 6 5]
 [3 3 2]]
[[False False False]
 [ True  True  True]]
```

They will work on arrays of any size and shape.

WORKING WITH BOOLEAN ARRAYS

- Boolean arrays → array with True and False as values.
- False is interpreted as 0.
- True is interpreted as 1.
- They can be counted for the entire array, for each row / column, etc – using ufunc by specifying the criterion.

61

WORKING WITH BOOLEAN ARRAYS

- COUNTING ENTRIES

- The True / False entries can be counted – using ufunc by specifying the criterion.

```
import numpy as np
x = np.random.randint(10, size=(2, 3))
print(np.count_nonzero(x < 6))
print(np.sum(x < 6))
```

Output:

```
[[9 6 5]
 [3 3 2]]
4
4
```

- This summation can be done along rows / columns.

```
# how many values less than 6 in
each row?
print(np.sum(x < 6, axis=1))
```

```
[1  3]
```

62

WORKING WITH BOOLEAN ARRAYS

- COUNTING ENTRIES

- To check whether any or all the values are True use
 - `np.any()` or `np.all()`

```
import numpy as np
x = np.random.randint(10, size=(2, 3))
print(np.any(x < 6)) # any value < 6?
print(np.all(x > 6)) # all values > 6?
print(np.any(x < 5, axis=1)) # any
values less than 5 in each row?
print(np.all(x > 2, axis=0)) # all values
greater than 2 in each column?
```

Output:

```
[[9 6 5]
 [3 3 2]]
True
False
[False True]
[True True False]
```

WORKING WITH BOOLEAN ARRAYS

- BOOLEAN OPERATORS

- To check how many values satisfy 2 or more criteria.
- Eg. how many values are between 3 and 7?

```
import numpy as np
x = np.random.randint(10, size=(2, 3))
print(x)
print(np.sum((x > 3) & (x < 7)))
print(np.sum(np.bitwise_and((x > 3), (x < 7))))
```

Output:

```
[[9 6 5]
 [3 3 2]]
2
2
```

Boolean Operators	Equivalent Ufunc
&	<code>np.bitwise_and</code>
	<code>np.bitwise_or</code>
^	<code>np.bitwise_xor</code>
~	<code>np.bitwise_not</code>

WORKING WITH BOOLEAN ARRAYS

- BOOLEAN ARRAYS AS MASKS

- Masking → To select particular subsets of the data
- Use Boolean arrays as masks

```
import numpy as np
x = np.random.randint(20, size=(3, 5))
print(x)
print(x > 5)
y = x[x < 10]
print(y)
```

Output:

```
[[ 7  4 15  2  4]
 [17 13 11  8  7]
 [ 2  5 16 15 10]]
[[ True False  True False False]
 [ True  True  True  True  True]
 [False False  True  True  True]]
[7 4 2 4 8 7 2 5]
```

65



FANCY INDEXING

FANCY INDEXING

- **EXPLORING FANCY INDEXING**

- CREATING A 1-D ARRAY FROM 1-D ARRAY
- CREATING A 2-D ARRAY FROM 1-D ARRAY
- CREATING A 1-D ARRAY FROM 2-D ARRAY
- CREATING A 2-D ARRAY FROM 2-D ARRAY

- **COMBINED INDEXING**

- COMBINING FANCY INDEXING AND SIMPLE INDEXING
- COMBINING FANCY INDEXING AND SLICING
- COMBINING FANCY INDEXING AND MASKING

- **MODIFYING VALUES WITH FANCY INDEXING**

67

FANCY INDEXING

- Accessing and modify portions of arrays using simple indices and slicing.
- Another style of array indexing → fancy indexing.
- Fancy indexing
 - Passing an array of indices to access multiple array elements at once.
 - Similar to the simple indexing arrays of indices but arrays of indices are passed instead of single scalars
 - Allows for **non-sequential** selection of elements
 - Allows to quickly access and modify complicated subsets of an array's values.
 - Always creates a **copy** of the data, (but basic slicing provides a view into the original array)

68

EXPLORING FANCY INDEXING

- Fancy indexing → passing an array of indices to access multiple array elements at once. For eg. To access three different elements we can pass a single list or array of indices.
- An 1-d array can be created using different elements from an 1-d array

```
import numpy as np
x = np.random.randint(100, size = 10)
print(x)
print([x[3], x[7], x[2]])
ind = [3, 7, 2]
print(x[ind])
y = x[[3, 7, 2]]
print(y)
```

Output:

```
[51 92 14 71 60 20 82 86 74 74]
[71, 86, 14]
[71 86 14]
[71 86 14]
```

EXPLORING FANCY INDEXING

- A 2-d array can be created from elements at chosen indices from a 1-d array – indices passed as a 2-d array

```
import numpy as np
rand = np.random.RandomState(42)
x = rand.randint(100, size=10)
print(x)
ind = np.array([[3, 7], [4, 5]])
print(x[ind])
```

Output:

```
[51 92 14 71 60 20 82 86 74 74]
[[71 86]
 [60 20]]
```

EXPLORING FANCY INDEXING

- A 1-d array can be created from elements at chosen indices from a 2-d array – indices passed as a 2-d array

```
import numpy as np
x = np.arange(20, 32).reshape((3, 4))
print(x)
row = np.array([0, 1, 2])
col = np.array([2, 1, 3])
print(x[row, col])
```

Output:

```
[[20 21 22 23]
 [24 25 26 27]
 [28 29 30 31]]

[22 25 31]
```

Elements $x[0,2]$, $x[1,1]$, $x[2,3]$ are formed into a new array

71

EXPLORING FANCY INDEXING

- A 2-d array can be created from elements at chosen indices from a 2-d array – indices passed as a 2-d array

```
import numpy as np
x = np.arange(12).reshape((3, 4))
print(x)
row = np.array([0, 1, 1])
col = np.array([2, 1, 3])
y = x[row[:, np.newaxis], col]
print("Y = ", y)
```

Output:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Y = [[ 2  1  3]
      [ 6  5  7]
      [ 6  5  7]]
```

- The full rows 0, 1, 1 are considered, and columns 2, 1, 3 are considered
- $[[x[0,2], x[0,1], x[0,3]], [x[1,2], x[1,1], x[1,3]], [x[1,2], x[1,1], x[1,3]]]$
- Shape of the indices is broadcasted

72

EXPLORING FANCY INDEXING

```
row = np.array([0, 1, 1])
col = np.array([2, 1, 3])
y = x[row[:, np.newaxis], col]
```

Explanation

- Here row array's shape becomes (3, 1) → $\begin{bmatrix} [0] \\ [1] \\ [1] \end{bmatrix}$
- Hence row = $\begin{bmatrix} [0] \\ [1] \\ [1] \end{bmatrix}$ and col = $\begin{bmatrix} [2] \\ [1] \\ [3] \end{bmatrix}$
- The full rows 0, 1, 1 are considered, and columns 2, 1, 3 are considered
- Now y has $\begin{bmatrix} [x[0,2], x[0,1], x[0,3]] \\ [x[1,2], x[1,1], x[1,3]] \\ [x[1,2], x[1,1], x[1,3]] \end{bmatrix}$
- Shape of the indices is broadcasted

73

EXPLORING FANCY INDEXING

- A 2-d array can be created from elements at chosen indices from a 2-d array – indices passed as a 2-d array

```
import numpy as np
x = np.arange(12).reshape((3, 4))
print(x)
row = np.array([0, 1, 1])
col = np.array([2, 1, 3])
y = x[row, col[:, np.newaxis]]
print("Y == ", y)
```

Output:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Y == [[ 2  6  6]
      [ 1  5  5]
      [ 3  7  7]]
```

- The full rows 0, 1, 1 are considered, and columns 2, 1, 3 are considered
- $\begin{bmatrix} [x[0,2], x[1,2], x[1,2]] \\ [x[0,1], x[1,1], x[1,1]] \\ [x[0,3], x[1,3], x[1,3]] \end{bmatrix}$
- Shape of the indices is broadcasted

74

EXPLORING FANCY INDEXING

```
row = np.array([0, 1, 1])
col = np.array([2, 1, 3])
y = x[row, col[:, np.newaxis]]
```

Explanation

- Here col array's shape becomes (3, 1)
- Hence row = [0 1 1] and col = $\begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}$
- The full rows 0, 1, 1 are considered, and columns 2, 1, 3 are considered
- Now y has $[[x[0,2], x[0,1], x[0,3]], [x[1,2], x[1,1], x[1,3]], [x[1,2], x[1,1], x[1,3]]]$
- Shape of the indices is broadcasted

75

COMBINED INDEXING

- Fancy and simple indices can be combined.

```
import numpy as np
x = np.arange(50, 62).reshape((3, 4))
print(x)
print(x[2, [2, 0, 1]])
```

Output:

```
[[ 50  51  52  53]
 [ 54  55  56  57]
 [ 58  59  60  61]]
[60  58  59]
```

- The 2nd row is considered, and columns 2, 0, 1 are considered
- $[x[2,2], x[2,0], x[2,1]]$

76

COMBINED INDEXING

- Fancy indexing and slicing can be combined.

```
import numpy as np
x = np.arange(50, 62).reshape((3, 4))
print(x)
print(x[1 :, [2, 0, 1]])
```

Output:

```
[[ 50  51  52  53]
 [ 54  55  56  57]
 [ 58  59  60  61]]
```

```
[[56  54  55]
 [60  58  59]]
```

- The rows from 1st row (1st and 2nd rows) are considered, and columns 2, 0, 1 are considered
- `[x[1,2], x[1, 0,], x[1, 1]]` → 1st row
- `[x[2,2], x[2, 0,], x[2, 1]]` → 2nd row

77

COMBINED INDEXING

- Fancy indexing and masking can be combined.

```
import numpy as np
x = np.arange(60, 72).reshape((3, 4))
print(x)

row = np.array([0, 2, 1])
col = np.array([1, 0, 2, 0])
print(x[row[:, np.newaxis], col])
```

Output:

```
[[ 60    61    62    63]
 [ 64    65    66    67]
 [ 68    69    70    71]]
```

```
[[61  60    62    60]
 [69  68    70    68]
 [65  64    66    64]]
```

78

MODIFYING VALUES WITH INDEXING

```
import numpy as np
x = np.arange(10, 20)
print(x)

i = np.array([2, 1, 5, 9])
x[i] = 99
print(x)

x[i] -= 5
print(x)
```

Output:

```
[10 11 12 13 14 15 16 17 18 19]
```

```
[10 99 99 13 14 99 16 17 18 99]
```

```
[10 94 94 13 14 94 16 17 18 94]
```

79

MODIFYING VALUES WITH INDEXING

```
import numpy as np
x = np.zeros(10)
x[[0, 5]] = [4, 6]
print(x)

x = np.ones(10)
x[[0, 5]] = [4, 6]
x[[7, 7]] = [33, 88]
print(x)
```

Output:

```
[4. 0. 0. 0. 0. 6. 0. 0. 0. 0.]
```

```
[ 4.  1.  1.  1.  1.  6.  1. 88.  1.  1.]
```

80

MODIFYING VALUES WITH INDEXING

```
import numpy as np
x = np.ones(10)
print(x)
i = [2, 3, 3, 4, 4, 4]
x[i] += 1
print(x)

np.add.at(x, i, 1)
print(x)
```

Output:

[1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]

[1. 1. 2. 2. 2. 1. 1. 1. 1. 1.]

[1. 1. 3. 4. 5. 1. 1. 1. 1. 1.]

81



SORTING ARRAYS

SORTING ARRAYS

- **FAST SORTING IN NUMPY:
NP.SORT AND NP.ARGSORT**
 - SORTING ALONG ROWS OR COLUMNS
- **PARTIAL SORTS: PARTITIONING**
- **EXAMPLE: K-NEAREST NEIGHBORS**

83

SORTING ARRAYS

- Selection sort → simple → but much too slow to be useful for larger arrays.
- Complexity → $O(n^2)$

```
import numpy as np
x = np.array([2, 1, 4, 3, 5])
for i in range(len(x)):
    swap = i + np.argmin(x[i:])
    (x[i], x[swap]) = (x[swap], x[i])
print(x)
```

84

SORTING ARRAYS

- Sorting algorithm based on generates and test para-diagram
- Bogo sort is a highly inefficient and impractical sorting algorithm.
- Bogo sort, the algorithm repeatedly shuffles the elements in the list randomly and checks if the list is sorted.
- This process continues until the list happens to be sorted by chance, which could take an extremely long time
- It's not a recommended sorting method for any real-world applications.
- Bogo Sort also known as permutation sort, stupid sort, slow sort, shotgun sort or monkey sort BogoSort also known as permutation sort, stupid sort, slow sort, shotgun sort

85

SORTING ARRAYS

- Bogo sort → relies on pure chance → it repeatedly applies a random shuffling of the array until the result happens to be sorted
- np.any does element-wise comparison on both the slices. If it returns true (any element in slice1 > its corresponding element in the slice2), the array is shuffled; If it returns false, it means array is sorted.
- Complexity → $O(n \times n!)$ → so not suited for any real computation

```
import numpy as np
x = np.array([2, 1, 4, 3, 5])
while np.any(x[:-1] > x[1:]):
    np.random.shuffle(x)
print(x)
```

How this works

$x[:-1]$ = [2 1 4 3]

↓ ↓ ↓ ↓

$x[1:]$ = [1 4 3 5]

True (because 2 > 1 and 4 > 3) So shuffled.

FAST SORTING

- Efficient built-in sorting algorithms available in Python
- Fast Sorting in NumPy: `np.sort` and `np.argsort`
- By default `np.sort` uses an $N \log N$, quicksort algorithm, though mergesort and heapsort are also available.



87

FAST SORTING

- `np.sort` → sorted version of the array without modifying the input
- `sort` → to sort the array in-place

```
import numpy as np
x = np.array([2, 1, 4, 3, 5])
y = np.sort(x)
print("X = ", x)
print("Y = ", y)

x.sort()
print("X = ", x)
```

Output:

```
X = [2, 1, 4, 3, 5]
Y = [ 1  2  3  4  5 ]

X = [ 1  2  3  4  5 ]
```

88

FAST SORTING

- `np.argsort` → returns the indices of the sorted elements
- For eg., in original array, 1 is in 1st position, 2 is in 0th position, 3 is in 3rd position, 4 is in 2nd position and 5 is in 4th position

```
import numpy as np
x = np.array([2, 1, 4, 3, 5])
pos = np.argsort(x)
print(pos)

print(x[pos])
```

Output:

```
[ 1 0 3 2 4 ]
```

```
[ 1 2 3 4 5 ]
```

89

SORTING ALONG ROWS OR COLUMNS

- NumPy's sorting algorithms has the ability to sort along specific rows or columns of a multidimensional array using the `axis` argument.

```
import numpy as np
rand = np.random.RandomState(42)
X = rand.randint(0, 10, (4, 6))
print(X)
cc = np.sort(X, axis=0) # column-wise
rr = np.sort(X, axis=1) # row-wise
print(cc, rr)
```

Output: X

```
[[6 3 7 4 6 9]
```

```
[2 6 7 4 3 7]
```

```
[7 2 5 4 1 7]
```

```
[5 1 4 0 9 5]]
```

cc

```
[[2 1 4 0 1 5]
```

```
[5 2 5 4 3 7]
```

```
[6 3 7 4 6 7]
```

```
[7 6 7 4 9 9]]
```

rr

```
[[3 4 6 6 7 9]
```

```
[2 3 4 6 7 7]
```

```
[1 2 4 5 7 7]
```

```
[0 1 4 5 5 9]]
```

90

PARTIAL SORTS: PARTITIONING

- Want to find the K smallest values in the array? → use `np.partition` function

Using `np.partition` in 1-d array

- The result is a new array has 2 partitions - with the smallest K values in the 1st partition, and the 2nd partition in no specific order.

```
import numpy as np
x = np.array([4, 2, 3, 1, 7, 5, 6])
y = np.partition(x, 3)
print(y)
```

Output:
[2 1 3 4 7 5 6]

91

PARTIAL SORTS: PARTITIONING

Using `np.partition` in 2-d array

- When **k = 2** and **axis = 1**, the result is an array where the first **two slots** in each row contain the smallest values from that **row**, with the remaining values filling the remaining slots.

Output: X
[[6 3 7 4 6 9]
[2 6 7 4 3 7]
[7 2 5 4 1 7]
[5 1 4 0 9 5]]

```
import numpy as np
rand = np.random.RandomState(42)
X = rand.randint(0, 10, (4, 6))
print(X)
Y = np.partition(X, 2, axis=1)
print(Y)
```

Output: Y
[[3 4 6 7 6 9]
[2 3 4 7 6 7]
[1 2 4 5 7 7]
[0 1 4 5 9 5]]



STRUCTURED DATA

STRUCTURED DATA

- **NUMPY'S STRUCTURED ARRAYS**
- **CREATING STRUCTURED ARRAYS**
- **MORE ADVANCED COMPOUND TYPES**
- **RECORDARRAYS: STRUCTURED ARRAYS WITH A TWIST**

NUMPY'S STRUCTURED ARRAYS

- Sometimes efficient storage for compound, heterogeneous data
- Eg., several categories of data on a number of people can be stored in 3 separate arrays → but a clumsy approach

```
name = ['Alice', 'Bob', 'Cathy', 'Doug']
age = [25, 45, 37, 19]
weight = [55.0, 85.5, 68.0, 61.5]
```

95

NUMPY'S STRUCTURED ARRAYS

- Better solution → single structure to store all of this data.
- NumPy can handle this through structured arrays, which are arrays with compound data types

```
data = np.zeros(4, dtype={'names':('name', 'age', 'weight'),
'formats':('U10', 'i4', 'f8')}) # empty container array created
print(data.dtype)
```

- 'U10' translates to "Unicode string of maximum length 10," 'i4' translates to "4-byte (i.e., 32 bit) integer," and 'f8' translates to "8-byte (i.e., 64 bit) float."

Output:

```
[('name', '<U10'), ('age', '<i4'), ('weight', '<f8')]
```

NUMPY'S STRUCTURED ARRAYS

- Now the array can be filled with the lists of values (name, age, weight)

```
data = np.zeros(4, dtype={'names':('name', 'age', 'weight'),
'formats':('U10', 'i4', 'f8')})      # empty container array created
print(data.dtype)
data['name'], data['age'], data['weight'] = name, age, weight
print(data)
```

Output:

```
[('name', '<U10'), ('age', '<i4'), ('weight', '<f8')]
[('Alice', 25, 55.) ('Bob', 45, 85.5) ('Cathy', 37, 68. )
 ('Doug', 19, 61.5)]
```

NUMPY'S STRUCTURED ARRAYS

- Now the data can be arranged in structured arrays and can be accessed easily

```
print(data['name'])
print(data[0])
print(data[-1])
print(data[-1]['name'])
```

```
print(data[data['age'] > 30] ['name'])
```

Output:

```
['Alice' 'Bob' 'Cathy' 'Doug']
('Alice', 25, 55.)
('Doug', 19, 61.5)
Doug
```

```
['Bob' 'Cathy']
```

- Pandas package can be used to do any operations that are any more complicated than these.
- Pandas provides a DataFrame object → a structure built on NumPy arrays

CREATING STRUCTURED ARRAYS

- Method-1: Dictionary method

```
data = {'names':('name', 'age', 'weight'), 'formats':('U10', 'i4', 'f8')}
print(np.dtype(data))
```

Output:

```
[('name', '<U10'), ('age', '<i4'), ('weight', '<f8')]
```

99

CREATING STRUCTURED ARRAYS

- Method-2: As a list of tuples

```
data = [('name', 'S10'), ('age', 'i4'), ('weight', 'f8')]
print(np.dtype(data))
```

```
data1 = ('S10, i4, f8')          # if column (field) names not known
print(np.dtype(data1))
```

Output:

```
[('name', 'S10'), ('age', '<i4'), ('weight', '<f8')]
```

```
[('f0', 'S10'), ('f1', '<i4'), ('f2', '<f8')]
```

CREATING STRUCTURED ARRAYS

```
[('name', 'S10'), ('age', '<i4'), ('weight', '<f8')]
```

```
[('f0', 'S10'), ('f1', '<i4'), ('f2', '<f8')]
```

- Shortened string format codes have 3 characters:
 1. < or > → represents little endian or big endian → specifies the ordering convention for significant bits
 2. S or I or f → specifies the type of data
 3. 10 or 4 or 8 → represents the size of the object in bytes.

101

CREATING STRUCTURED ARRAYS

NumPy data types:

Character	Description	Example
'b'	Byte	<code>np.dtype('b')</code>
'i'	Signed integer	<code>np.dtype('i4')</code>
'u'	Unsigned integer	<code>np.dtype('u1')</code>
'f'	Floating point	<code>np.dtype('f8')</code>
'c'	Complex floating point	<code>np.dtype('c16')</code>
'S', 'a'	string	<code>np.dtype('S5')</code>
'U'	Unicode string	<code>np.dtype('U') =</code>
'V'	Raw data (void)	<code>np.dtype('V') =</code>

102

ADVANCED COMPOUND TYPES

- NumPy structured data type with two fields – an ‘id’ and a ‘mat’ component consisting of a 3×3 floating-point matrix

```
import numpy as np
cols = [('id', 'i8'), ('mat', 'f8', (3, 3))]    # 2 fields created
X = np.zeros(2, dtype = np.dtype(cols)) # an array with 2 rows created
print(X)    # full array with 2 rows each with 2 fields
print(X['mat'][0])    # 'mat' field of 0th row
```

Output:

```
[(0, [[0., 0., 0.], [0., 0., 0.], [0., 0., 0.]])
 (0, [[0., 0., 0.], [0., 0., 0.], [0., 0., 0.]])
 [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
```

ADVANCED COMPOUND TYPES

- Values assigned to ‘mat’ in the form of a matrix ‘val’

```
import numpy as np
val = [[8,6,3],[1,7,5],[0,3,9]]
cols = [('id', 'i8'), ('mat', 'f8', (3, 3))]
X = np.zeros(2, dtype = np.dtype(cols))
print(X)
print("\n")
X['mat']= val
print(X)
```

Output:

```
[(0, [[0., 0., 0.], [0., 0., 0.], [0., 0., 0.]])
 (0, [[0., 0., 0.], [0., 0., 0.], [0., 0., 0.]])
 [(0, [[8., 6., 3.], [1., 7., 5.], [0., 3., 9.]])
 (0, [[8., 6., 3.], [1., 7., 5.], [0., 3., 9.]])]
```

RECORD ARRAYS - STRUCTURED ARRAYS WITH A TWIST

- Array fields can be accessed as attributes – using `np.recarray`

```
import numpy as np
name = ['Alice', 'Bob', 'Cathy', 'Doug']
age = [25, 45, 37, 19]
weight = [55.0, 85.5, 68.0, 61.5]

data = np.zeros(4, dtype={'names':('name', 'age', 'weight'),
'formats':('U10', 'i4', 'f8')})      # empty container array created
data['name'], data['age'], data['weight'] = name, age, weight
print(data)
data_rec = data.view(np.recarray)
print(data_rec.age)
```

RECORD ARRAYS - STRUCTURED ARRAYS WITH A TWIST

- Array fields can be accessed as attributes – using `np.recarray`

Output:

```
[('Alice', 25, 55. ) ('Bob', 45, 85.5) ('Cathy', 37, 68. ) ('Doug', 19, 61.5)]
[25 45 37 19]
```

- **Methods to create arrays using random number generator**

- `float_arr1 = np.random.random(10)`
float random numbers
- `float_arr2 = np.random.uniform(10, size=5)`
float random numbers
- `int_arr = np.random.randint(30, size=10)`
int random numbers

-

- **Methods to create arrays with specific values**

- `a = np.array([4, 9, 1, 8, 10, 2])`
user defined array
- `b = np.ones((3, 3))`
2-d array of size 9 with all values = 1.0
- `c = np.ones((3, 3), dtype='int')`
2-d array of size 9 with all values = 1
- `d = np.zeros((8))`
array of size 8 with all values = 0
- `e = np.arange(30)`
1-d array of size 30 with a sequence of values

Ufuncs:

- **Methods for aggregations**

- `np.sum(float_arr1)`
- `np.min(float_arr1)`
- `np.max(float_arr1)`
- `np.argmin(arr)`

Other methods on arrays

- `np.shape(b)`
- `np.reshape(3, 2)`
- `np.count_nonzero(x < 6)`
- `print(np.any(x < 6))`
- `print(np.all(x > 6))`
- `np.sort(x)`
- `np.argsort(x)`
- `np.partition(x, 4)`
- `np.rearrange`

107



108